RED BLACK TREE

Project Description

Data Structures – C Programming

1. Project Overview

A Red Black Tree (RBT) is a self-balancing Binary Search Tree in which every node has a color, Red or Black. The color rules help keep the tree approximately balanced during insertion and deletion. Rotations and recoloring are used to restore the Red Black Tree properties after modifications.

2. Objective

The objective of this project is to implement a Red Black Tree in C and provide the required operations for inserting, deleting, searching, finding minimum and maximum values, deleting minimum and maximum values, and displaying the tree.

3. Red Black Tree Properties

- Each node is either Red or Black.
- The root is Black.
- All NIL leaves are Black.
- If a node is Red, both of its children are Black.
- Every path from a node to descendant NIL nodes contains the same number of Black nodes.

4. Operations Implemented

Operation	Description
Insert	Inserts a data item and restores Red Black Tree balance using recoloring and rotations.
Delete	Deletes a data item and performs Red Black deletion fix-up when required.
Search	Searches for a specified data item.
Find Minimum	Finds the minimum value in the tree.
Delete Minimum	Deletes the minimum value from the tree.
Find Maximum	Finds the maximum value in the tree.
Delete Maximum	Deletes the maximum value from the tree.
Print Tree	Displays tree data and node colors using inorder traversal.

5. Project Files

File	Purpose
main.c	Menu-driven main program and user interaction.
rbt.h	Tree structure, color definitions, constants, and prototypes.
insert.c	Insertion, node creation, rotations, and insertion

	balancing.
delete.c	Deletion and Red Black deletion fix-up.
search.c	Search operation.
find_min.c	Find minimum operation.
find_max.c	Find maximum operation.
delete_min.c	Delete minimum operation.
delete_max.c	Delete maximum operation.
print_tree.c	Inorder display of tree data and colors.

6. Technologies and Concepts Used

- C Programming
- Data Structures
- Binary Search Trees
- Red Black Trees
- Pointers
- Structures
- Dynamic Memory Allocation
- Tree Traversal
- Tree Rotations
- Recoloring and Balancing

7. Compilation and Execution

Compile the complete project using:

```
gcc -Wall -Wextra main.c insert.c delete.c search.c delete_min.c delete_max.c find_min.c find_max.c print_tree.c -o rbt
```

Run the executable using:

```
./rbt
```

8. Menu Options

- 1. Insert
- 2. Delete
- 3. Search
- 4. Find Minimum
- 5. Find Maximum
- 6. Delete Minimum
- 7. Delete Maximum
- 8. Print Tree
- 9. Exit

9. Expected Outcome

The application provides a menu-driven interface for all required Red Black Tree operations. The tree is maintained according to the Red Black Tree balancing rules, and the display operation shows stored values with their node colors.

10. Conclusion

This project demonstrates the implementation of a self-balancing Red Black Tree using C. It applies pointers, structures, dynamic memory allocation, tree traversal, rotations, recoloring, insertion, deletion, searching, and minimum/maximum operations.

11. Reference

Emertxe Information Technologies (P) Ltd., Requirements & Design Document – Red Black Tree, Data Structures Module.